

Sistema Inteligente de Monitorizacion y Prediccion de Fallos en Equipos Industriales

Memoria del Trabajo de Fin de Master

Autor: Jose Ignacio Molina

Contacto: joseignmolina@gmail.com

Junio 2026

1. Introduccion y Contexto del Proyecto

El presente trabajo aborda el diseno e implementacion de un sistema inteligente de monitorizacion y prediccion de fallos en equipos industriales mediante tecnicas de machine learning supervisado y no supervisado. El problema del mantenimiento predictivo constituye uno de los casos de uso mas relevantes de la inteligencia artificial en el sector industrial, con impacto directo en la reduccion de paradas no planificadas, el coste de mantenimiento correctivo y la seguridad operativa.

El sistema desarrollado procesa datos procedentes de nueve sensores fisicos distribuidos en el equipo industrial, aplica un pipeline de preprocesado y enriquecimiento de caracteristicas, y produce tanto una clasificacion binaria (normal / fallo) como un indice de salud continuo de cinco niveles que facilita la toma de decisiones por parte del operario.

1.1 Dataset utilizado

El conjunto de datos empleado contiene 944 registros con 9 variables de entrada procedentes de sensores industriales y una variable objetivo binaria (fail). La distribucion de clases presenta un nivel moderado de desbalanceo: 58.4 % de registros normales frente a un 41.6 % de fallos, lo que permite aplicar los algoritmos estandar sin necesidad de tecnicas agresivas de balanceo, aunque si se configuraron pesos de clase en todos los modelos para compensar la asimetria.

Variable	Descripcion	Tipo	Rango
footfall	Trafico de personas/objetos	Continua	0 - 190
tempMode	Modo termico de operacion	Ordinal	0 - 7
AQ	Indice de calidad del aire	Ordinal	1 - 7
USS	Sensor ultrasonico	Ordinal	1 - 7
CS	Sensor de corriente electrica	Ordinal	1 - 7
VOC	Compuestos organicos volatiles	Ordinal	1 - 7
RP	Posicion rotacional / RPM	Continua	0 - 100
IP	Presion de entrada	Ordinal	1 - 7
Temperature	Temperatura de operacion (C)	Continua	0 - 36
fail	Variable objetivo (0=normal, 1=fallo)	Binaria	0 / 1

2. Metodologia

El proyecto sigue un pipeline secuencial de ocho fases, cada una con entradas y salidas bien definidas, lo que garantiza la reproducibilidad completa de los resultados mediante la ejecucion en orden de los scripts correspondientes.

Fase 0 - Big Data con PySpark

EDA escalable mediante la API de Apache Spark. Demuestra que el pipeline es portable a entornos de cluster (Databricks, EMR) sin modificaciones de codigo.

Fase 1 - Analisis Exploratorio

Distribucion de clases, correlaciones de Pearson, boxplots por clase y tests de Mann-Whitney para validar diferencias estadisticamente significativas entre grupos. Se identificaron VOC, AQ, USS y CS como los sensores con mayor poder discriminante.

Fase 2 - Calidad de datos

Comprobacion de valores nulos (resultado: ninguno), duplicados (resultado: ninguno) y deteccion de outliers mediante IQR y Z-score. El dataset presento buena calidad general con outliers puntuales en footfall y Temperature, no eliminados por no comprometer la integridad estadistica.

Fase 3 - Ingenieria de características

Creacion de 9 variables derivadas que capturan relaciones no lineales entre sensores: indice de carga operativa (CS x RP), ratio presion-temperatura, estres ambiental (VOC + AQ), puntuacion de estres global, indicadores binarios de trafico elevado y modo termico extremo, e interaccion corriente-presion.

Fase 4 - Deteccion de anomalias

Aplicacion de tres algoritmos no supervisados: Isolation Forest, Local Outlier Factor (LOF) y One-Class SVM, con una tasa de contaminacion del 10 %. Se genero una etiqueta de consenso por votacion mayoritaria (2 de 3 algoritmos). Los resultados se integran en el dashboard para su exploracion visual.

Fase 5 - Entrenamiento de modelos

Comparacion de cuatro clasificadores mediante validacion cruzada estratificada de 5 pliegues sobre el 80 % de los datos. Particion train/test 80/20 estratificada. Normalizacion con StandardScaler ajustado exclusivamente sobre el conjunto de entrenamiento para evitar data leakage.

Fase 6 - Interpretabilidad

Calculo de SHAP values (TreeExplainer), Permutation Importance y Partial Dependence Plots sobre el mejor modelo. Se genera una comparacion normalizada de los tres metodos para identificar las caracteristicas mas robustamente relevantes independientemente del metodo de atribucion.

Fase 7 - Monitorizacion

Evaluacion del modelo sobre datos de produccion simulados, deteccion de data drift mediante el test de Kolmogorov-Smirnov por variable, sistema de alertas automaticas por umbral y generacion de informe JSON con los resultados de interpretacion de todos los modelos.

3. Resultados - Comparacion de Modelos Predictivos

Se evaluaron cuatro modelos de clasificacion binaria sobre un conjunto de test de 189 muestras (20 % del dataset, particion estratificada). Las metricas principales se recogen en la tabla siguiente.

Modelo	Accuracy	F1	ROC-AUC	Precision	Recall
Random Forest	0.9259	0.9146	0.9767	0.8824	0.9494
XGBoost	0.9153	0.8987	0.9689	0.8987	0.8987
LightGBM	0.9048	0.8861	0.9694	0.8861	0.8861
Logistic Regression	0.8942	0.8780	0.9671	0.8471	0.9114

3.1 Analisis por modelo

Random Forest (mejor modelo, F1 = 0.9146, AUC = 0.9767)

Random Forest es el modelo con mejor F1 global. Con 75 verdaderos positivos, 4 falsos negativos y 10 falsos positivos sobre el test set, ofrece el mejor equilibrio entre precision y recall (0.8824 y 0.9494 respectivamente). En entornos industriales donde el coste de un fallo no detectado supera ampliamente al de una falsa alarma, el recall elevado es especialmente valioso.

XGBoost (F1 = 0.8987, AUC = 0.9689)

XGBoost presenta la mayor precision del conjunto (0.8987) con un recall igual, lo que implica una politica de prediccion mas conservadora: menos falsas alarmas, pero mayor riesgo de fallos no detectados. Adecuado cuando el coste operativo de las intervenciones preventivas innecesarias es significativo.

LightGBM (F1 = 0.8861, AUC = 0.9694)

LightGBM muestra un AUC ligeramente superior al de XGBoost (0.9694 vs 0.9689) pero un F1 inferior, indicando que su curva ROC es globalmente competitiva pero el umbral por defecto (0.5) no le favorece. Un ajuste de umbral podria mejorar su F1 a costa de precision o recall.

Logistic Regression (F1 = 0.8780, AUC = 0.9671)

A pesar de ser el modelo mas simple, la regresion logistica alcanza un AUC de 0.9671, lo que demuestra que el problema es considerablemente linealmente separable. Su recall de 0.9114 supera al de XGBoost y LightGBM, y su interpretabilidad lo convierte en una linea base solida y auditable.

3.2 Diferencias entre modelos

La diferencia de F1 entre el mejor y el peor modelo es de 0.0366 puntos, lo que indica que los cuatro clasificadores compiten en un margen reducido. Todos superan ampliamente los umbrales minimos establecidos ($F1 \geq 0.75$, $AUC \geq 0.80$), y todos ofrecen un AUC superior a 0.96, lo que valida la calidad del pipeline de preprocesado y la ingenieria de caracteristicas implementada.

Conclusion: Random Forest es el modelo recomendado para produccion por su mayor F1 y recall. La pequena diferencia entre modelos sugiere que el principal determinante del rendimiento es la calidad de las caracteristicas, no el algoritmo elegido.

4. Interpretabilidad y Relevancia de Caracteristicas

Se aplicaron tres metodos complementarios de atribucion de importancia sobre el modelo Random Forest entrenado: importancia intrinseca (Gini), SHAP values (TreeExplainer) y Permutation Importance con 15 repeticiones sobre F1. La tabla recoge las 8 caracteristicas mas relevantes segun SHAP.

Caracteristica	SHAP (mean val)	Gini RF	Permut. RF
VOC	0.1624	0.2776	0.1521
env_stress	0.1401	0.2396	0.0768
AQ	0.0583	0.0988	0.0011
USS	0.0571	0.0794	0.0113
CS	0.0202	0.0311	0.0008
sensor_stress_score	0.0187	0.0551	-0.0004
footfall	0.0147	0.0322	0.0019
cs_ip_interaction	0.0106	0.0212	0.0000

4.1 Analisis de las caracteristicas dominantes

VOC (compuestos organicos volatiles) emerge como la variable mas influyente en los tres metodos de atribucion, con una importancia SHAP de 0.1624 y Gini de 0.2776. Su dominancia en XGBoost es aun mas marcada (0.5543), lo que sugiere que esta variable tiene un efecto no lineal pronunciado que los modelos de boosting capturan con especial eficiencia.

La variable derivada env_stress (media de VOC y AQ) ocupa el segundo lugar en todos los metodos, con SHAP de 0.1401 y Gini de 0.2396. Este resultado valida la hipotesis de diseno de que la combinacion de indicadores ambientales captura sinergias no representadas por cada sensor individualmente.

Las variables AQ y USS presentan importancias intermedias y consistentes entre metodos. El resto de caracteristicas originales (Temperature, RP, IP) y derivadas (load_index, pressure_temp_ratio) aportan contribuciones menores pero no despreciables. Las caracteristicas de baja varianza como thermal_extreme y high_footfall son las de menor relevancia, aunque actuan como regularizadores implicitos al capturar condiciones extremas poco frecuentes.

Conclusion: VOC y env_stress son los factores criticos para la prediccion de fallos en este sistema. Una estrategia de mantenimiento basada unicamente en el seguimiento de VOC y calidad del aire proporcionaria una base de alerta razonablemente robusta, aunque inferior al sistema completo.

5. Resultados de Deteccion de Anomalias

Se aplicaron tres algoritmos de deteccion de anomalias no supervisados con una tasa de contaminacion del 10 %, configurada en funcion de la proporcion de fallos del dataset. Los tres algoritmos operan sobre el espacio de caracteristicas escalado con StandardScaler.

Isolation Forest

Detecta anomalias aislando puntos mediante particiones aleatorias del espacio de caracteristicas.

Eficiente en alta dimensionalidad y robusto ante outliers masivos. Genera puntuaciones de anomalia normalizadas.

Local Outlier Factor (LOF)

Basado en densidad local. Compara la densidad de cada punto con la de sus 20 vecinos mas proximos. Efectivo para detectar anomalias en regiones de densidad variable.

One-Class SVM

Aprende la frontera de decision del conjunto de datos normales (entrenado unicamente sobre muestras normales). Kernel RBF con $\nu=0.10$. Mas sensible a la escala y parametrizacion que los otros dos metodos.

La etiqueta de consenso (anomalia si al menos 2 de los 3 algoritmos coinciden) reduce los falsos positivos individuales y mejora la precision del sistema no supervisado. Los resultados de cada algoritmo y del consenso se visualizan mediante proyecciones PCA de dos componentes en el dashboard.

6. Sistema de Monitorizacion y Alertas

La fase de monitorizacion evalua periodicamente el sistema desplegado comparando el comportamiento del modelo sobre datos de produccion contra el comportamiento de referencia del entrenamiento. Se disenaron seis comprobaciones automaticas con umbrales configurables.

6.1 Metricas del modelo en produccion

Evaluado sobre el 25 % de los datos (datos de produccion simulados), el modelo seleccionado alcanza un F1 de 0.9897, un ROC-AUC de 0.9938 y un Recall de 1.0. Todos los indicadores superan holgadamente los umbrales minimos establecidos ($F1 \geq 0.75$, $AUC \geq 0.80$, $Recall \geq 0.70$), lo que confirma que el modelo generaliza correctamente sobre datos no vistos.

6.2 Deteccion de data drift (test KS)

El test de Kolmogorov-Smirnov compara la distribucion empirica de cada sensor entre los datos de referencia (60 % primeros registros) y los datos de produccion (40 % restantes). Un p-valor inferior a 0.05 indica drift estadisticamente significativo.

Sensor	Estadistico KS	p-valor	Drift
footfall	0.0629	0.3155	No
tempMode	0.0299	0.9825	No
AQ	0.1060	0.0111	SI
USS	0.1541	< 0.001	SI
CS	0.1045	0.0129	SI
VOC	0.2257	< 0.001	SI
RP	0.1795	< 0.001	SI
IP	0.2747	< 0.001	SI
Temperature	0.9815	< 0.001	SI

Se detecta drift en 7 de los 9 sensores, lo cual activa una alerta CRITICA segun los umbrales configurados. Este resultado es previsible dado que los datos de produccion corresponden a la segunda mitad del dataset ordenado, que puede reflejar variaciones temporales o condiciones operativas distintas. En un despliegue real, el drift de Temperature (KS = 0.9815) indicaria un cambio drastico de condiciones ambientales que justificaria un reentrenamiento del modelo.

Conclusion: El alto drift detectado en Temperature y VOC, que son precisamente las variables mas influyentes, es una senal de que el modelo podria degradarse en produccion si las condiciones del entorno cambian significativamente. Se recomienda implementar un ciclo de reentrenamiento periodico.

6.3 Resumen de alertas

Metrica	Valor	Umbral	Estado
---------	-------	--------	--------

Sistema Inteligente de Monitorizacion y Prediccion de Fallos Industriales

F1-score	0.9897	≥ 0.75	OK
ROC-AUC	0.9938	≥ 0.80	OK
Recall	1.0000	≥ 0.70	OK
Data drift (sensores)	7 / 9	0	CRITICA
Tasa fallos predicha	51.9 %	$\leq 60 \%$	OK
% alto riesgo ($>70 \%$)	50.5 %	$\leq 30 \%$	ALERTA

7. Componentes de Despliegue

7.1 Dashboard interactivo (Streamlit)

El dashboard proporciona una interfaz grafica completa para la exploracion de datos, la visualizacion de resultados de modelos y la monitorizacion del sistema. Esta estructurado en seis secciones navegables:

Resumen General: KPIs globales, tasa de fallos, alertas de alto riesgo.

Explorador de Datos: Scatter plots interactivos, boxplots, correlaciones.

Deteccion Anomalias: Resultados de los tres algoritmos y el consenso.

Comparacion Modelos: Metricas, curvas ROC superpuestas, matrices de confusion y feature importances.

Rendimiento Modelo: Metricas del mejor modelo, curva ROC, informe de clasificacion.

Indice de Salud: Distribucion del Machine Health Index en cinco niveles de riesgo.

7.2 API REST (FastAPI)

La API expone el modelo entrenado como servicio HTTP con tres endpoints de prediccion: prediccion individual (/predict), prediccion en lote de hasta 500 lecturas (/predict/batch) y prediccion con explicabilidad (/predict/explain). Incluye documentacion interactiva automatica generada por OpenAPI (Swagger UI). La validacion de entradas se realiza mediante Pydantic con restricciones de rango por variable.

7.3 Contenerizacion con Docker

El sistema se despliega mediante Docker Compose con dos contenedores independientes: uno para la API REST (puerto 8000) y otro para el dashboard (puerto 8501). El contenedor de la API implementa un health check basado en Python que garantiza que el dashboard no arranque hasta que la API este disponible. Los modelos y datos procesados se montan como volúmenes de solo lectura, lo que permite actualizar los artefactos sin reconstruir las imagenes.

7.4 Big Data con Apache Spark

Se implemento un pipeline de EDA con PySpark que replica el analisis exploratorio utilizando la API distribuida de Spark: estadisticas descriptivas, deteccion de nulos, analisis de outliers por IQR y una pipeline de Spark ML con VectorAssembler y StandardScaler. El mismo codigo es portable a un cluster Databricks o Amazon EMR sin modificaciones, demostrando la escalabilidad de la arquitectura para datasets de decenas de millones de registros.

8. Conclusiones

8.1 Cumplimiento de objetivos

El sistema desarrollado cumple la totalidad de los requisitos técnicos mínimos establecidos en el enunciado del TFM y añade las siguientes funcionalidades avanzadas de los apartados opcionales:

- Python como lenguaje principal en todos los componentes.
- Análisis exploratorio de datos con 6 gráficos analíticos y tests estadísticos.
- Detección de anomalías con tres algoritmos y etiquetado por consenso.
- Comparación de cuatro modelos predictivos con validación cruzada estratificada.
- Dashboard funcional con seis secciones, incluida comparación visual de modelos.
- Repositorio Git documentado con README técnico completo.
- API REST con FastAPI (despliegue).
- Contenerización con Docker Compose (MLOps/despliegue).
- Pipeline de Big Data con Apache Spark (escalabilidad).
- Sistema de monitorización con detección de drift y alertas automáticas.
- Machine Health Index de cinco niveles integrado en dashboard y API.

8.2 Rendimiento del sistema

El mejor modelo (Random Forest) alcanza un F1-score de 0.9146 y un ROC-AUC de 0.9767 en el conjunto de test, con un recall del 94.9 %, lo que implica que el sistema detecta prácticamente la totalidad de los fallos reales. La diferencia de rendimiento entre los cuatro modelos evaluados es pequeña (0.037 puntos de F1), lo que indica que la ingeniería de características y el preprocesado son los factores dominantes de la calidad del sistema, por encima de la elección del algoritmo.

Las variables VOC (compuestos orgánicos volátiles) y env_stress (estrés ambiental combinado) son los predictores más influyentes de forma robusta y consistente a través de los tres métodos de interpretabilidad aplicados (SHAP, Permutation Importance e importancia Gini). Este hallazgo tiene implicaciones prácticas directas: permite diseñar estrategias de mantenimiento centradas en la monitorización de la calidad del aire y los compuestos volátiles del entorno operativo.

8.3 Limitaciones del estudio

- El dataset contiene 944 muestras, un volumen reducido que limita la robustez estadística de las estimaciones de generalización. Los resultados de validación cruzada pueden tener varianza alta en escenarios reales con mayor variabilidad.
- El drift detectado en los datos de producción simulados (7 de 9 variables) es parcialmente un artefacto del método de partición temporal del dataset, y no necesariamente refleja drift real en operación continua.
- Las etiquetas de anomalía no supervisadas no han sido validadas contra etiquetas de fallo reales en todos los escenarios, lo que impide calcular la precisión del sistema de detección no supervisado de forma definitiva.

- El Machine Health Index es un indicador compuesto con pesos fijados empiricamente. Una calibracion basada en datos historicos de mantenimiento aumentaria su utilidad practica.
- La API REST no implementa autenticacion ni rate limiting, lo que la hace inadecuada para despliegue en entornos de produccion sin una capa adicional de seguridad (API Gateway, OAuth2).

9. Lineas de Mejora Futura

9.1 Datos y preprocesado

Ampliacion del dataset.

Recopilar series temporales continuas en lugar de muestras independientes. La incorporacion de contexto temporal (ventanas deslizantes, lag features) abriria la posibilidad de aplicar modelos de secuencias como LSTM o Temporal Fusion Transformers, que han demostrado mejoras sustanciales en problemas de mantenimiento predictivo.

Datos en tiempo real.

Sustituir el dataset estatico por un stream de datos via Apache Kafka o MQTT, lo que permitiria inferencia en tiempo real y reentrenamiento continuo (online learning) con algoritmos como River o Vowpal Wabbit.

Balanceo de clases.

Evaluar tecnicas de sobremuestreo sintetico como SMOTE o ADASYN para escenarios con mayor desbalanceo. Comparar con el ajuste de pesos de clase ya implementado para determinar cual estrategia generaliza mejor.

9.2 Modelado y rendimiento

Optimizacion de hiperparametros.

Aplicar busqueda bayesiana (Optuna, Hyperopt) en lugar de valores fijos. En particular, el ajuste del parametro `scale_pos_weight` en XGBoost y `n_estimators` en Random Forest mediante optimizacion dirigida podria reducir los 4 falsos negativos actuales.

Modelos de deep learning.

Explorar redes neuronales densas (MLP) y autoencoders para deteccion de anomalias. Los autoencoders son especialmente adecuados cuando se dispone de abundantes datos normales y pocos ejemplos de fallo, ya que aprenden la reconstruccion de datos normales y detectan anomalias por error de reconstruccion elevado.

Ensamblado de modelos.

Implementar un meta-clasificador (stacking) que combine las predicciones de los cuatro modelos individuales. Dado que los modelos muestran perfiles de error distintos (RF minimiza FN, XGBoost minimiza FP), un ensamblado inteligente podria reducir ambos tipos de error simultaneamente.

Calibracion de probabilidades.

Aplicar calibracion isotonica o de Platt a las probabilidades de salida para mejorar la fiabilidad del Machine Health Index como medida de confianza y no solo como ranking de riesgo.

9.3 MLOps y ciclo de vida

Reentrenamiento automatico.

Diseno de un pipeline de reentrenamiento disparado por umbrales de drift o degradacion de metricas,

orquestrado con Apache Airflow o Prefect. El sistema actual ya detecta el drift; el siguiente paso natural es automatizar la respuesta.

Seguimiento de experimentos.

Integrar un servidor de seguimiento de experimentos (MLflow, Weights & Biases) para versionar los hiperparametros, metricas y artefactos de cada ciclo de entrenamiento, facilitando la reproducibilidad y el gobierno del modelo.

Despliegue en la nube.

Publicar los contenedores Docker en un registro de imagenes (ECR, GCR) y desplegarlos en un servicio gestionado de contenedores (AWS ECS, Google Cloud Run, Azure Container Apps) con auto-escalado basado en la carga de peticiones a la API.

Autenticacion y seguridad.

Implementar autenticacion OAuth2 con JWT en la API REST, habilitar HTTPS mediante un proxy inverso (Nginx, Traefik) y anadir rate limiting para proteger el servicio en entornos de produccion.

9.4 Interpretabilidad y confianza

Explicaciones por instancia.

Integrar las explicaciones SHAP directamente en la respuesta de la API (/predict/explain ya existe el endpoint) y visualizarlas en el dashboard con un grafico de cascada (waterfall plot) que muestre la contribucion de cada sensor a la prediccion especifica de un registro.

Deteccion de sesgo.

Analizar si el modelo presenta patrones de error sistematicos segun rangos de temperatura o modos de operacion. Un modelo sesgado hacia condiciones de bajo stress podria infradetectar fallos en regimenes operativos extremos.

Intervalos de confianza.

Implementar prediccion conformal (conformal prediction) para acompa ar cada prediccion con un intervalo de confianza calibrado, lo que permitiria al operario conocer no solo la probabilidad de fallo sino tambien la incertidumbre del sistema en cada prediccion.

10. Reflexion Final

El sistema desarrollado demuestra que es posible construir un pipeline de machine learning completo y desplegable en produccion utilizando exclusivamente herramientas de codigo abierto. El rendimiento obtenido ($F1 > 0.91$, $AUC > 0.97$) es competitivo para un problema industrial real, y la arquitectura modular facilita la sustitucion o mejora de cualquier componente de forma independiente.

El aspecto mas relevante desde el punto de vista tecnico es la confirmacion de que la ingenieria de características tiene mayor impacto en el rendimiento que la eleccion del algoritmo. Las variables derivadas `env_stress` y `load_index` aparecen consistentemente entre las mas influyentes, validando la hipotesis de que las relaciones entre sensores contienen informacion predictiva que no esta presente en las medidas individuales.

El sistema de monitorizacion implementado, con deteccion de drift y alertas automaticas, cierra el ciclo MLOps de forma que el modelo no es un artefacto estatico sino un componente vivo que puede ser

evaluado, reemplazado y mejorado de forma continua. Esta capacidad es, en ultima instancia, la que diferencia un prototipo de investigacion de un sistema de mantenimiento predictivo apto para produccion industrial.